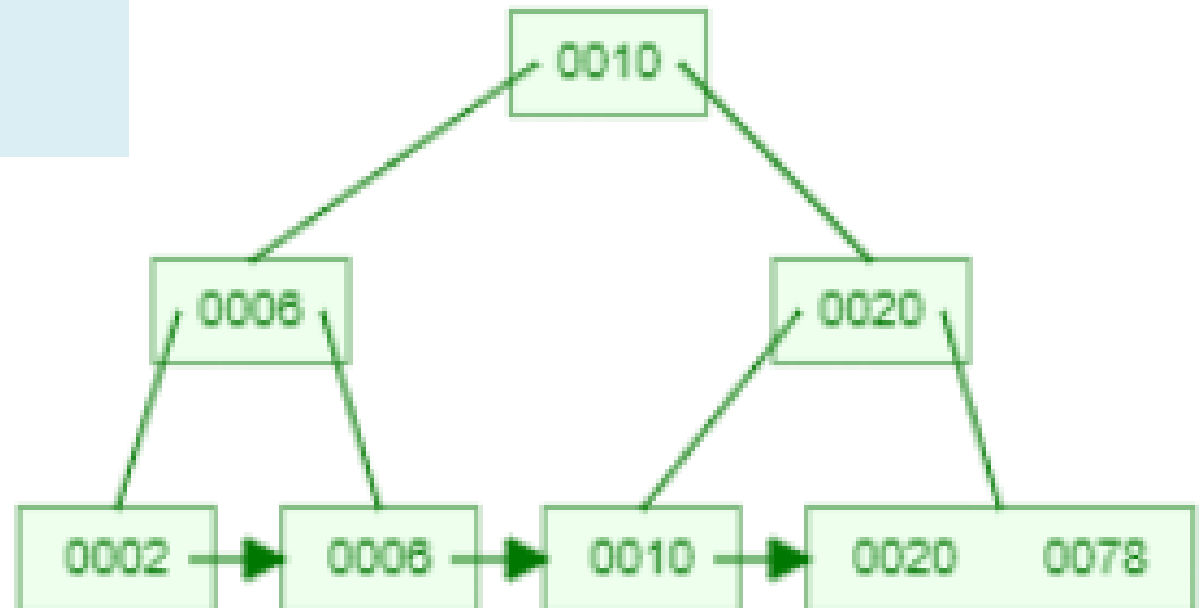


B+ Trees

Illustrative Example
of
Inserting a Node in a
B+ tree



B+ Trees

- In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.
- The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.
- B+ Tree are used to store the large amount of data which can not be stored in the main memory.

B+ Tree

B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations.

In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.

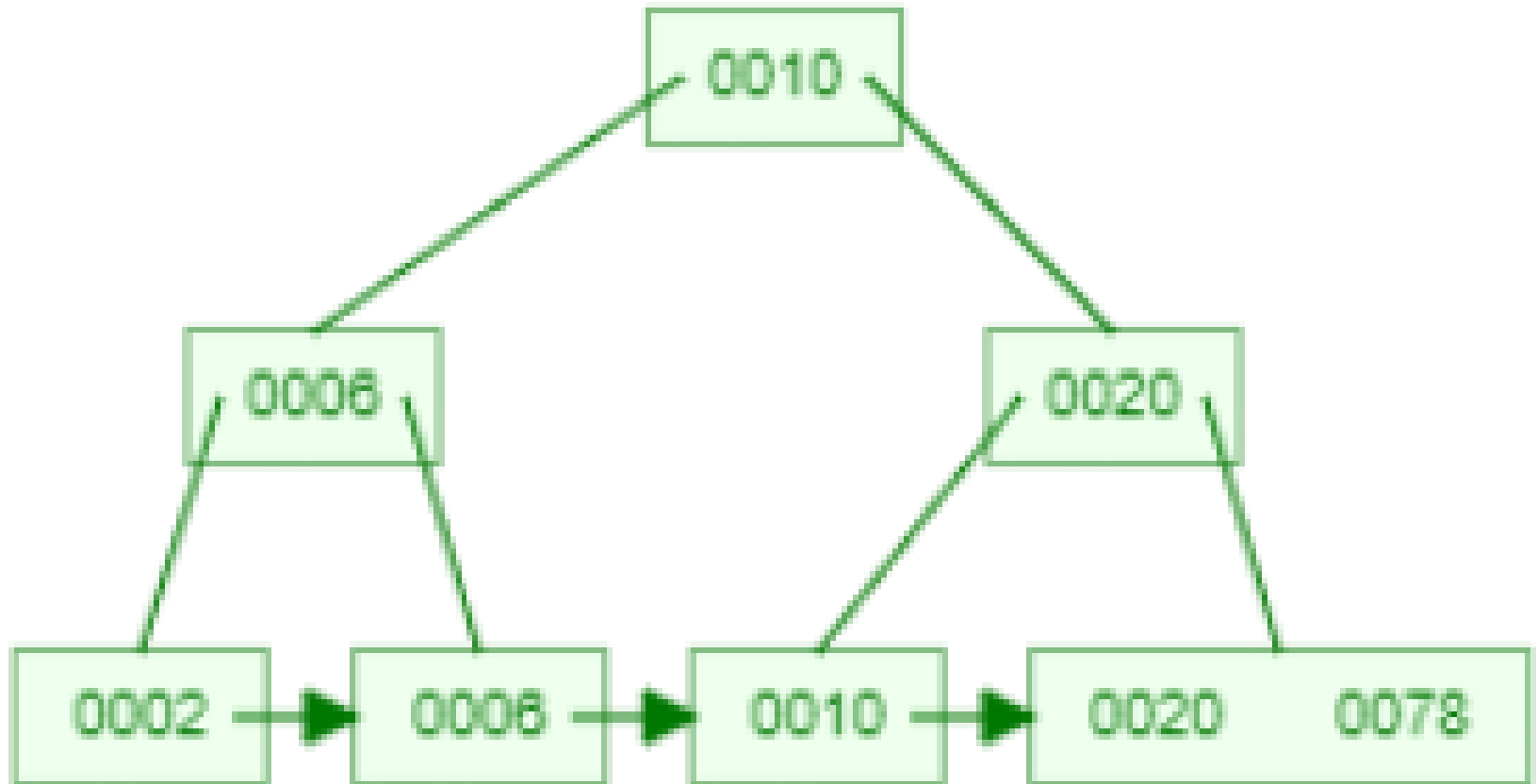
The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.

B+ Tree are used to store the large amount of data which can not be stored in the main memory.

Due to the fact that, size of main memory is always limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory whereas, leaf nodes are stored in the secondary memory

The internal nodes of B+ tree are often called index nodes.

A B+ tree of order 3 is shown in the following figure.



Insertion in B+ Tree

Step 1: Insert the new node as a leaf node

Step 2: If the leaf doesn't have required space, split the node and copy the middle node to the next index node.

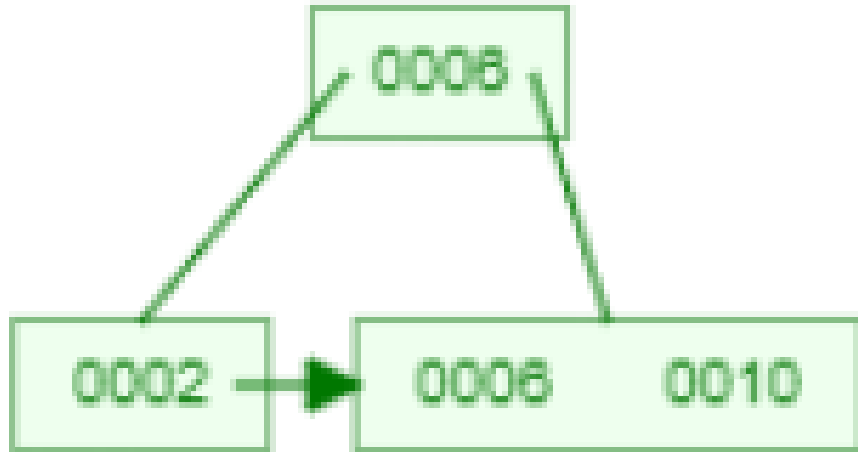
Step 3: If the index node doesn't have required space, split the node and copy the middle element to the next index page.

Insert 2, 6, 10, 20, 78, 5, 90 in B+ Tree of order 3

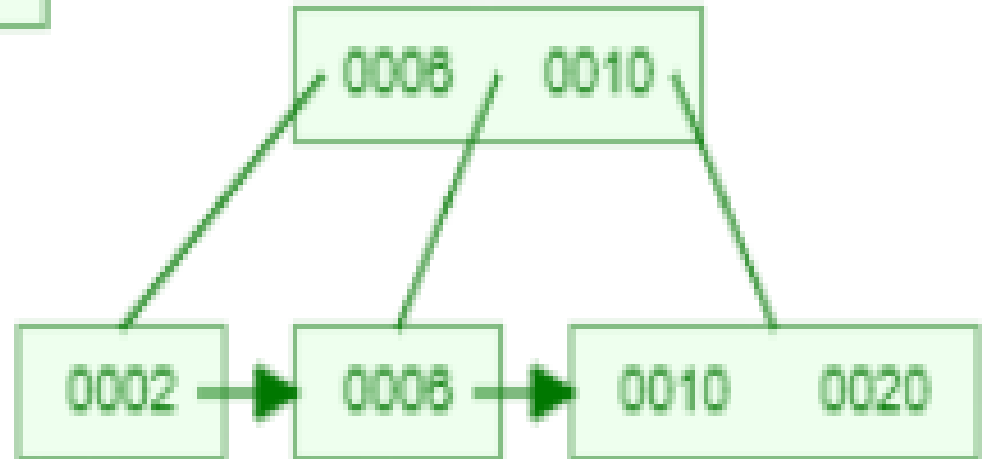


Insert 2, 6,

Insert 2, 6, 10, 20, 78, 5, 90,

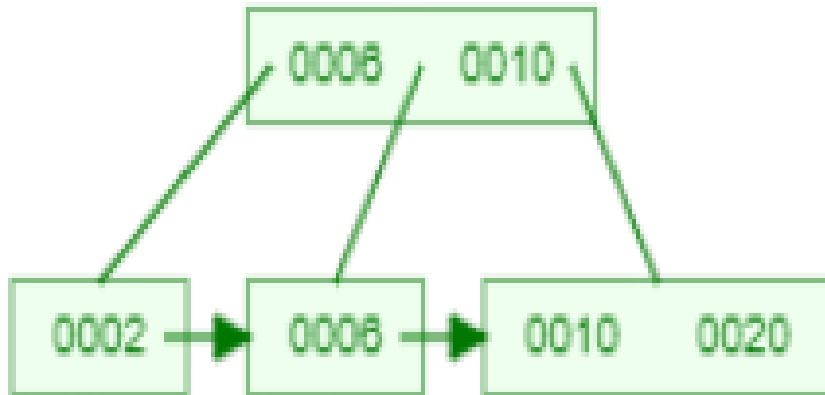


Insert 10

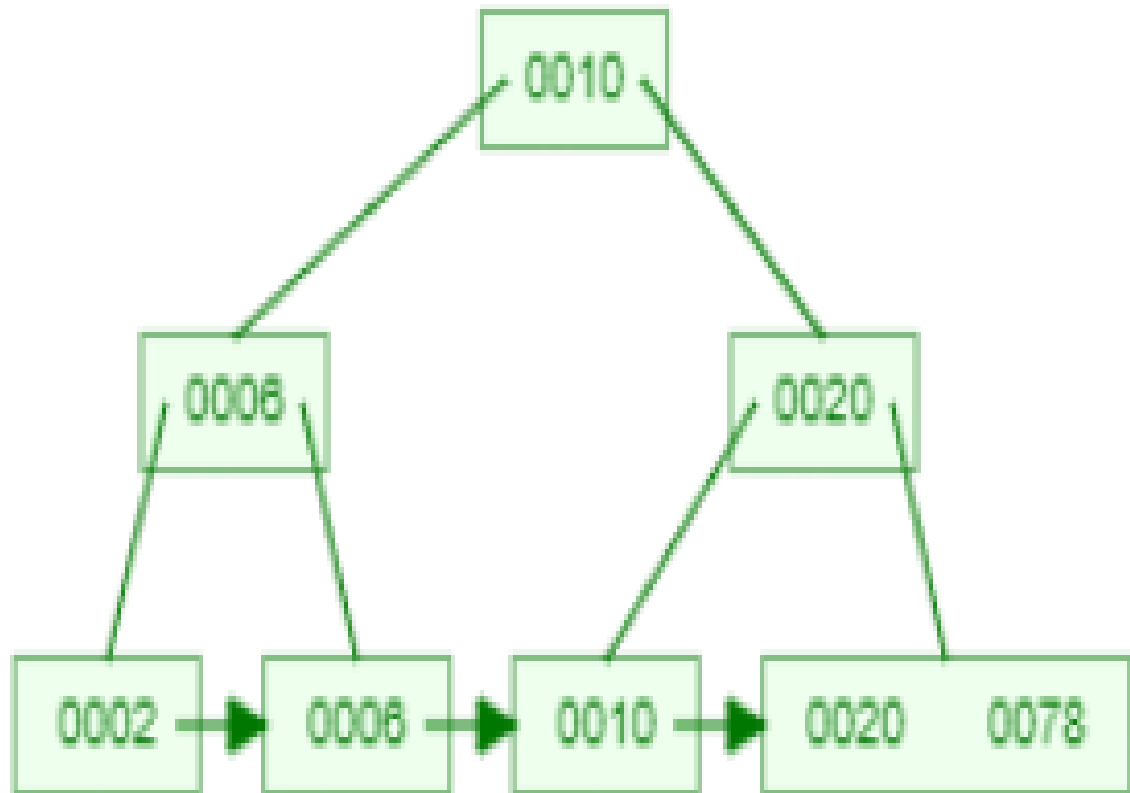


Insert 20

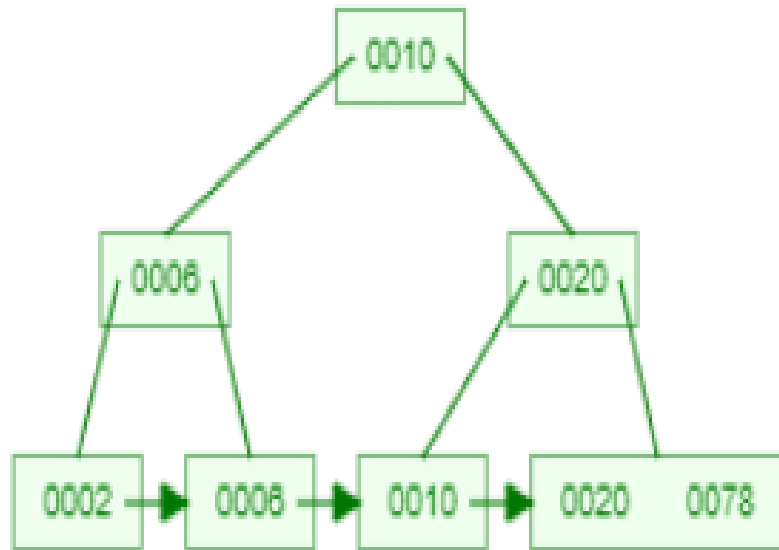
Insert 2, 6, 10, 20, 78, 5, 90,



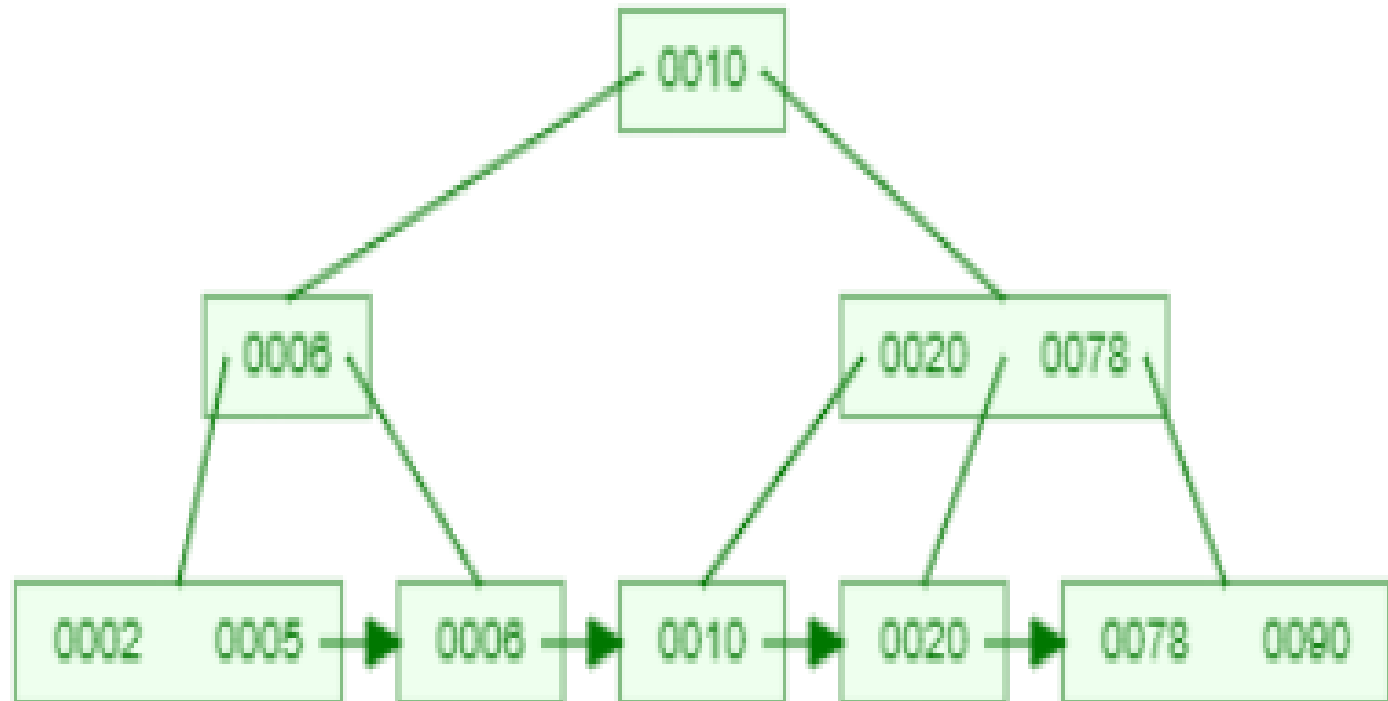
Insert 78



Insert 2, 6, 10, 20, 78, 5, 90,



Insert 5, 90

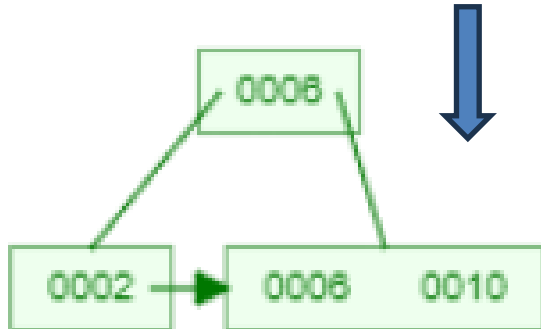


B+ Tree of order 3

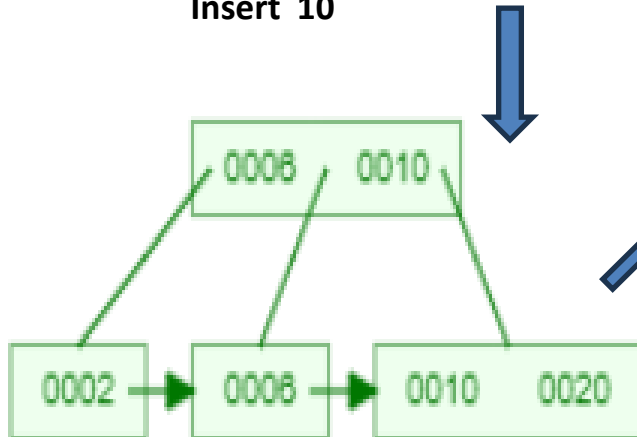
Insert 2, 6, 10, 20, 78, 5, 90,



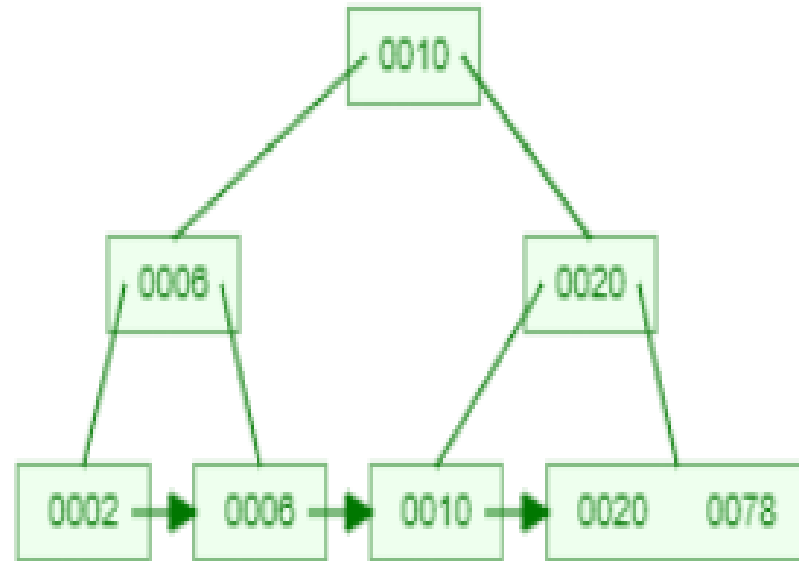
Insert 2, 6,



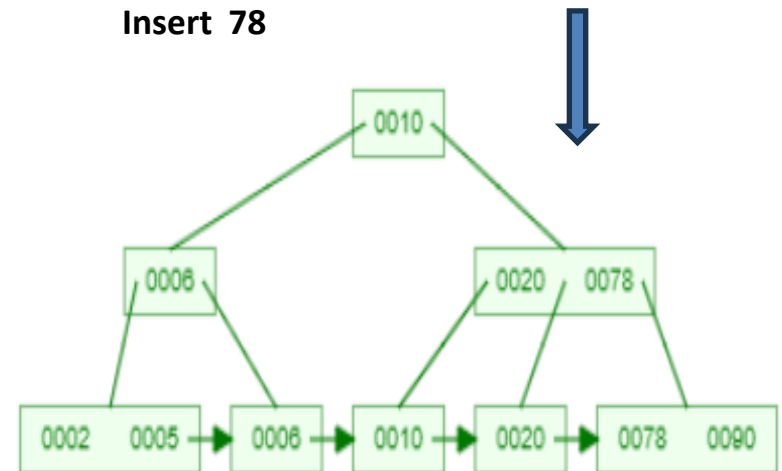
Insert 10



Insert 20



Insert 78



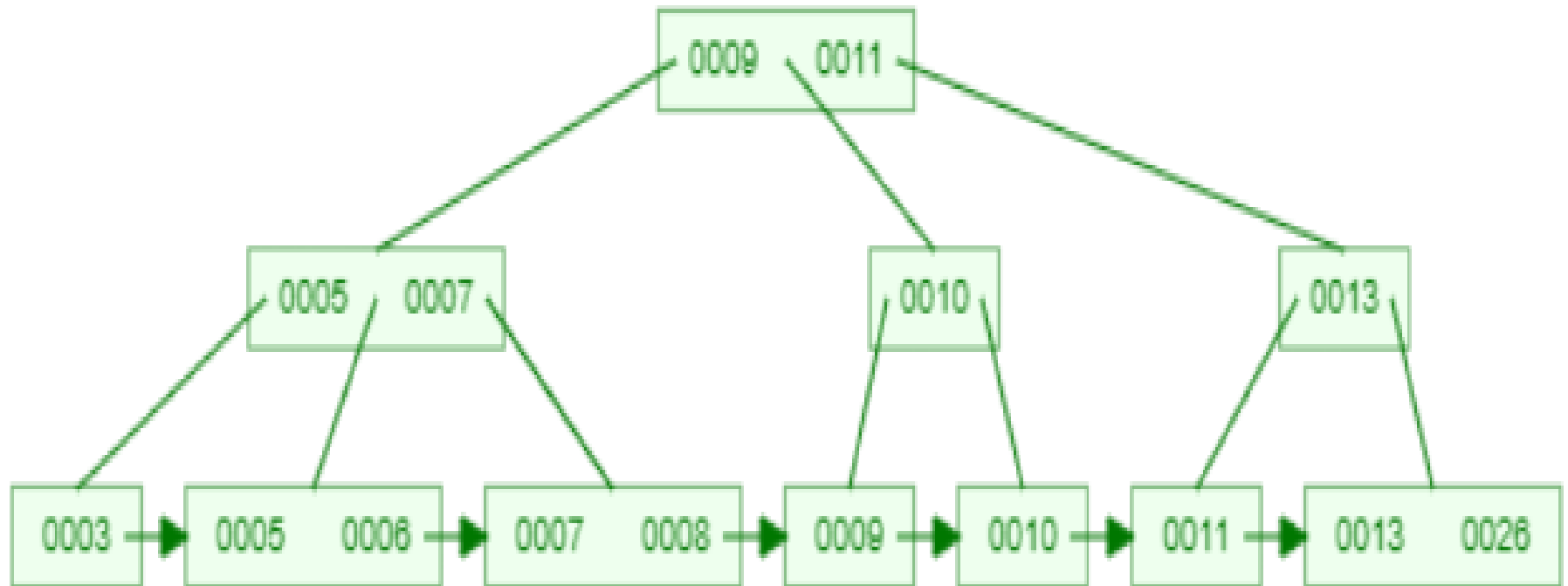
Insert 5, 90

B+ Tree of order 3

Insert 10, 9, 8, 11, 7, 5, 6, 26, 3, 13

B+ Tree of order 3

Insert 10, 9, 8, 11, 7, 5, 6, 26, 3, 13



Advantages of B+ Tree

- Records can be fetched in equal number of disk accesses.
- Height of the tree remains balanced.
- We can access the data stored in a B+ tree sequentially as well as directly.
- Keys are used for indexing.
- Faster search queries as the data is stored only on the leaf nodes.

B Tree VS B+ Tree

SN	B Tree	B+ Tree
1	Search keys can not be repeatedly stored.	Redundant search keys can be present.
2	Data can be stored in leaf nodes as well as internal nodes	Data can only be stored on the leaf nodes.
3	Searching for some data is a slower process since data can be found on internal nodes as well as on the leaf nodes.	Searching is comparatively faster as data can only be found on the leaf nodes.
4	Deletion of internal nodes are so complicated and time consuming.	Deletion will never be a complexed process since element will always be deleted from the leaf nodes.
5	Leaf nodes can not be linked together.	Leaf nodes are linked together to make the search operations more efficient.